Graph Analysis via Graph Rewriting: Sparse Linear Algebra Based Graph Analysis in Interaction Nets

Danil Zaripov[1,*], Efim Kubyshkin[1], Nikolai Ponomarev[1], Georgy Sichkar[1], Vladimir Zaikin[1] and Semyon Grigorev[1]

[1]*Saint Petersburg State University, 7-9 Universitetskaya Embankment, St. Petersburg, 199034, Russia*

Abstract

A clear and well-documented LaTeX document is presented as an article formatted for publication by CEUR-WS in a conference proceedings. Based on the "ceurart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

Keywords

Interaction Nets, graph rewriting, graph analysis algorithms, GraphBLAS, sparse linear algebra, quad-tree

1. Introduction

Graph analysis has become fundamental to modern data science, powering applications ranging from recommendation systems in social networks [?] and fraud detection in financial transactions [?] to protein interaction analysis in bioinformatics [?] and static code analysis [?]. As graph sizes grow to billions of vertices and edges, the need for efficient, scalable implementations becomes critical. However, graph analysis algorithms are difficult to parallelize due to their irregular memory access patterns, poor data locality, and load imbalance across computational units [?].

The GraphBLAS standard [1] addresses these challenges by providing a foundation for constructing high-performance parallel graph analysis algorithms. By expressing graph algorithms in the language of linear algebra—using sparse vectors and matrices as core data structures, together with a rich set of highly parameterizable operations (matrix multiplication, element-wise operations, etc.)—GraphBLAS offers a high-level, mathematically rigorous framework for algorithm construction. Moreover, these linear algebraic primitives can be optimized for diverse hardware architectures, providing a path to high-performance graph analysis across platforms. This approach has proven successful, with implementations like SuiteSparse:GraphBLAS [2] delivering high performance on shared-memory systems, and LAGraph [3] providing a collection of graph analysis algorithms built on these primitives. However, designing parallel functions over sparse data structures remains a nontrivial engineering challenge, particularly for accelerators. Beyond CPU implementations, GraphBLAS has been partially ported to such devices: GraphBLAST [4] and Spla [?] target GPGPUs, while GraphLily [5] targets FPGAs.

Despite its successes, GraphBLAS and its implementations face certain challenges. The algebraic structures central to GraphBLAS—such as semirings and monoids—align more naturally with functional programming paradigms than with imperative ones. Generic data structures, such as matrices and vectors, and functions that required to provide a flexible core for matrix processing, together with user-defined types and operations (which are themselves functions), complicate API design and usage, particularly with respect to type safety. This insight suggests that functional programming languages could simplify the design of GraphBLAS-like APIs, offering cleaner abstractions and more

Woodstock'22: Symposium on the irreproducible science, June 07–11, 2022, Woodstock, NY

*Corresponding author.

st129060@student.spbu.ru (D. Zaripov); e.kubyshkin@spbu.ru (E. Kubyshkin); n.ponomarev@spbu.ru (N. Ponomarev); georgy.sichkar@yandex.ru (G. Sichkar); st119057@student.spbu.ru (V. Zaikin); s.v.grigoriev@mail.spbu.ru (S. Grigorev)

0009-0003-1086-9279 (D. Zaripov); 0009-0009-5531-1758 (E. Kubyshkin); 0009-0000-2382-5687 (N. Ponomarev); 0009-0005-4684-9610 (G. Sichkar); 0009-0003-3417-990X (V. Zaikin); 0000-0002-7966-0698 (S. Grigorev)

© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

composable primitives. However, general-purpose functional languages are often not fast enough for high-performance numerical computing. Research into functional languages and frameworks for parallel computations (including GPUs)—such as Futhark [6], AnyDSL [7], LIFT [8]—while demonstrating promising results, has largely focused on dense computations, leaving sparse irregular workloads less explored. Special hardware for graph analysis. "A Survey on Graph Processing Accelerators: Challenges and Opportunities", "Graph Processing on FPGAs: Taxonomy, Survey, Challenges"

At the same time, interaction nets [9]—a graph-rewriting model introduced by Yves Lafont—offer a computational model that is inherently well-suited for irregular parallelism. In interaction nets, computation proceeds through local rewriting rules applied to pairs of connected nodes (so-called active pairs). Crucially, multiple non-interfering active pairs can be reduced simultaneously, making interaction nets a promising foundation for parallel irregular computations, including graph analysis. Furthermore, interaction nets are relatively close to functional programming languages, making it easy to transfer design approaches and ideas from the functional world.

In this paper, we present the design and implementation of a GraphBLAS-like linear algebra foundation in two different settings: F# (a mature functional-first language, chosen for ease of prototyping and testing) and Inpla (a language and runtime for interaction nets). We employ quadtrees to represent sparse matrices and vectors, enabling natural divide-and-conquer parallelism through recursive subdivision. On this foundation, we build three canonical graph algorithms—breadth-first search (BFS), single-source shortest paths (SSSP), and triangle counting—using the algebraic primitives provided by our API.

Our contributions are as follows.

1. A type-driven design for a GraphBLAS-like API, demonstrating how to utilize algebraic types together with a minimal set of operations (e.g., `map`, `map2`, `spmv`, `spgemm`) for natural linear-algebra-based graph and graph analysis algorithms encoding with compile-time safety.
2. Reference implementations of a minimal, functionally equivalent subset of the API in F#, Vine, and Inpla, enabling direct comparison of respective execution models and runtime systems.
3. A comprehensive empirical evaluation of BFS, SSSP, and triangle counting across the three implementations, including:
    - runtime and memory comparison against C-based GraphBLAS baselines;
    - performance scaling analysis on multi-core architectures.

The paper is structured as follows. Section 2 provides background on GraphBLAS, quadtree matrix representations, interaction nets, and the Inpla language. Section **??** details our API design and its implementation in both F# and Inpla. Section 3.2 describes the graph algorithms built on this API. Section 4 presents our experimental setup and evaluation results. Section **??** discusses related work, and Section 5 concludes with directions for future research.

2. Background

In this section we briefly discuss GraphBLAS and introduce tree-based matrix representation. We also provide the basics of interaction nets, discuss implementations of interaction nets, and motivate our choice of which one we use for evaluation.

2.1. GraphBLAS

The goal of the GraphBLAS API [1] is to provide basic building blocks for constructing high-performance graph analysis algorithms. To achieve this goal, GraphBLAS utilizes **sparse generic** linear algebra. Operations over sparse generic matrices and vectors parameterized by semiring-like structures provide a powerful basis for expressing a wide range of graph analysis algorithms [3].

Challenges on this path include the implementation of high-performance generic operations over sparse data structures. Imbalanced subtasks together with irregular data access patterns make load balancing — and thus efficient parallel implementation — hard. Moreover, graph analysis algorithms

are not single operations but nontrivial chains of operations over matrices and vectors. This leads to the problem of intermediate data structures. Kernel fusion could solve these problems, but it is too difficult to implement automatic optimizations for sparse generic computations, so it is done manually in the most frequent cases. Attempts to use MLIR (GraphBLAS dialect).!!! Poor utilization on GPUs. Nontrivial challenge to create generic kernels for GPU and FPGA.!!! There are also other problems discussed in "What Did GraphBLAS Get Wrong?" by John Gilbert[1]

SuiteSparse:GraphBLAS serves as the reference implementation of this API, offering a highly optimized, multi-threaded C library that supports various sparse matrix formats. Building on this foundation, LAGraph provides a collection of production-ready graph algorithms (e.g., PageRank, connected components, triangle counting) implemented using SuiteSparse:GraphBLAS. Currently it is one of the state-of-the-art (SOTA) solutions for high-performance graph analysis, thus we use it as a baseline for evaluation. GraphBLAS API: C API, C++ API, implementations in Java, Julia, etc.

The long-term goal of our work is to investigate how interaction nets can help solve challenges with irregular parallelism and, possibly, with kernel fusion. In this work, we do not focus on the GraphBLAS API implementation, but we are inspired by it: we implemented a minimal set of generic data structures and operations sufficient to implement several classic graph analysis algorithms.

2.2. Tree-based Representation for Sparse Matrices and Vectors

To store sparse matrices and vectors efficiently, we employ a quad-tree data structure. The matrix is recursively subdivided into four quadrants — northwest (NW), northeast (NE), southwest (SW), and southeast (SE) — until each block becomes either uniform or empty. The Figure 1 shows an example of a matrix in the quad-tree form.

This recursive partitioning allows large uniform blocks to be represented implicitly, thereby reducing the memory required to store the matrix. Unlike flat array formats such as CSR or CSC, which are standard in libraries like SuiteSparse:GraphBLAS, the quad-tree naturally supports a divide-and-conquer execution model and adapts dynamically to irregular sparsity patterns. This structure simplifies parallelization of element-wise, matrix-matrix, and matrix-vector operations, as independent sub-blocks can be processed recursively without global synchronization.

However, this flexibility comes at a cost: for dense or highly regular matrices, the overhead of maintaining tree metadata and pointer indirection can outweigh the benefits of sparse storage, making traditional linear formats more efficient in such cases.

2.3. Interaction Nets

Interaction Nets is a graph-rewriting computation model proposed by Y. Lafont in 1989 [9] and can be defined as follows.

Definition 1 (Agent)**.** *An agent is a symbol from a fixed alphabet Σ, for which:*

- *an arity is defined — a function $Ar : \Sigma \to \mathbb{N} \cup \{0\}$;*
- *there is one principal port and a number of auxiliary ports equal to its arity.*

Definition 2 (Net)**.** *A net over the alphabet Σ is a labeled undirected multi-graph in which:*

- *each node is labeled by an agent;*
- *edges can only exist between ports;*
- *at most one edge enters each port.*

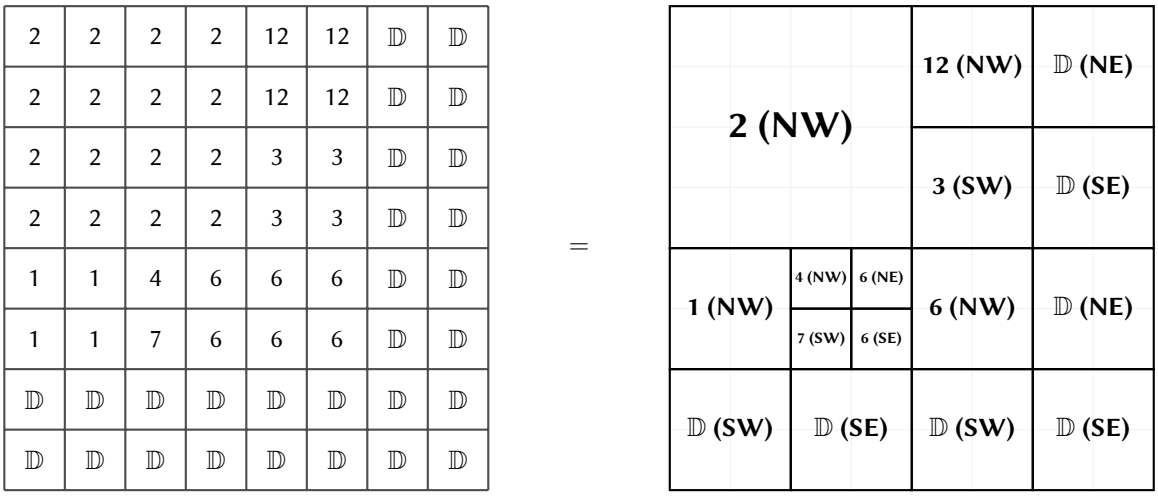
Let $\mathcal{N}_\Sigma$ denote the set of all possible nets over the alphabet Σ.

We say that port p_a is connected to port p_b if there is an edge between them. We say that a port is *free* if no edge originates from it. A net can also consist simply of a set of edges, in which case each end of an edge is considered a free port.

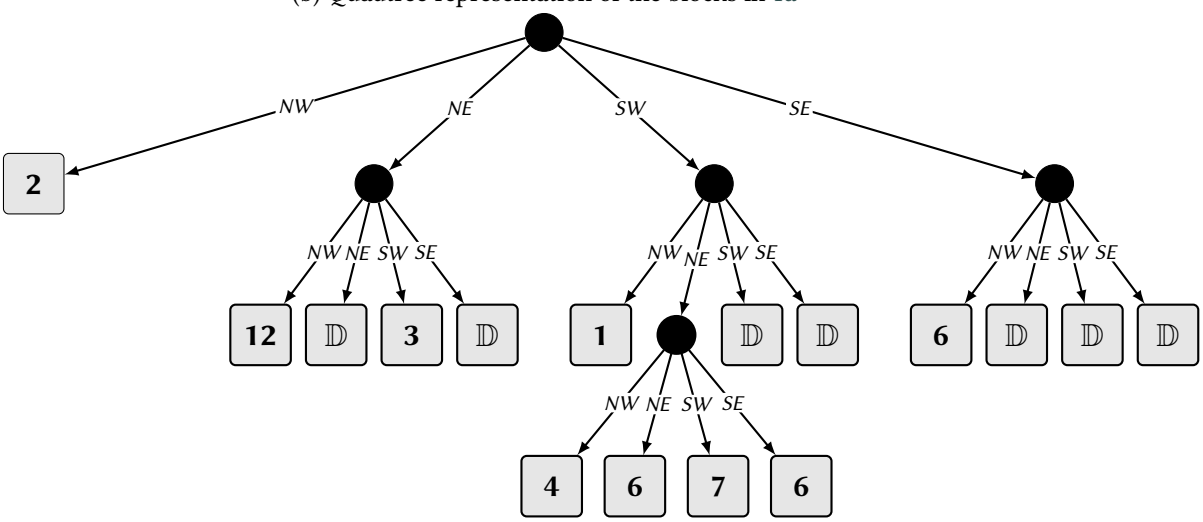
[1]Talk by John Gilbert, UC Santa Barbara, GraphBLAS BoF at HPEC, September 2022. Details can be found here: https://sites.cs.ucsb.edu/~gilbert/talks/talks.htm.

Figure 1: Example of the matrix and its quad-tree representation

(a) Block decomposition of the matrix



(b) Quadtree representation of the blocks in 1a



Definition 3 (Interface). *The interface $I(n)$ is the set of all free ports of a net n.*

Computation in Interaction Nets is defined in terms of *reductions*: rewriting an *active pair* into a net as defined in *reduction rules*. An active pair consists of two agents connected by an edge between their principal ports.

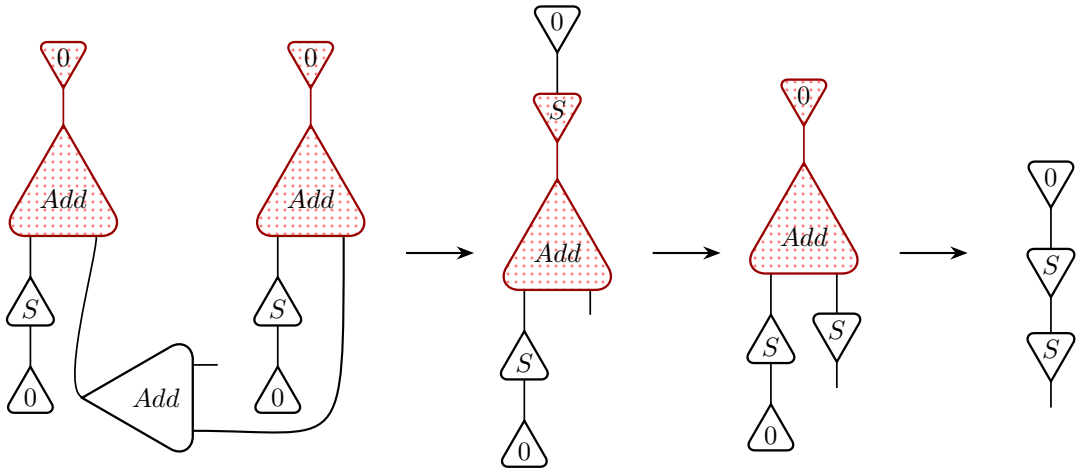
Definition 4 (Reduction Rule). *A reduction rule is a graph rewriting rule. Let R be the set of reduction rules. More formally, a reduction rule is a triplet $r \in \Sigma \times \Sigma \times \mathcal{N}_\Sigma$, for which the following properties hold:*

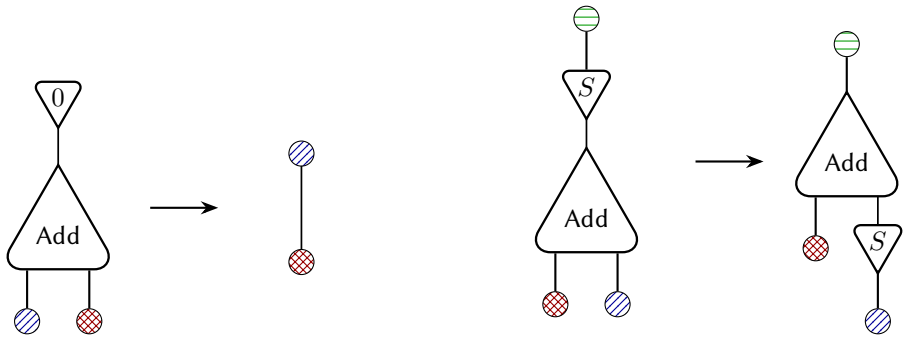
- ***Symmetry:*** $(l_1, l_2, n) \in R \Rightarrow (l_2, l_1, n) \in R$
- ***Determinism:*** $(l_1, l_2, n_1) \in R \wedge (m_1, m_2, n_2) \in R \Rightarrow (l_1, l_2) \neq (m_1, m_2)$
- ***Interface Preservation:*** $\forall (l_1, l_2, n) \in R, Ar(l_1) + Ar(l_2) = \|I(n)\|$

An example of the computation of the net encoding of the term $(0+1)+(0+1)$ is shown in Figure 2. The set of agents is $\Sigma = \{0, S, Add\}$ with arities $Ar(0) = 0$, $Ar(S) = 1$ and $Ar(Add) = 2$. Reduction rules shown in Figure 3. The left auxiliary ports of the active Add-agents are connected to the net representing 1, encoded in Peano numerals as $S(0)$, and the right auxiliary ports are connected to the

third Add-agent. There is one free port — one of the auxiliary ports of the Add-agent. This constitutes the interface.

Figure 2: Example of computation in interaction nets: reduction of the term $(1+0)+(1+0)$ encoded in Peano numerals





(a) Rule for $0 + x = x$

(b) Rule for $S(x) + y = S(x + y)$

Figure 3: Reduction rules for Peano numerals

As shown in Lafont's work [9], λ-calculus can be expressed through this formalism that makes interaction nets general purpose computation model. On the other hand, agents and reduction rules are not predefined, so one can define custom sets of agents and reduction rules, creating domain-specific (or problem-specific) models. Additionally, reductions can be performed independently, and it has been proven that their order does not matter. This is the primary reason for the natural suitability of Interaction Nets for parallel computation. These properties make Interaction Nets a promising platform for graph analysis tasks, as parallel graph analysis has an irregular nature. At the same time, if it is possible to introduce a powerful language for describing graph analysis algorithms (e.g., linear algebra), then accelerating the final solution may be possible by creating specific agents and rules.

2.4. Implementations of Interaction Nets

We are focused on languages and runtimes.

Notable implementations include Inpla, HVM, and Vine. In OCaml [10] (Comparison, Inpla fastest) Other projects implement embeddings interaction nets in languages such as OCaml and Haskell and use respective runtime to run.

Inpla project[2] [11] provides a low-level interaction nets programming language, equipped with a

[2]https://github.com/inpla/inpla

parallel runtime. Programming in Inpla requires defining a custom set of rewriting rules for a chosen set of agents. Computation begins when two agents are connected via their primary ports.

```
(* Defining custom rules *)
Add(r, x) >< Z => (* Inpla variant of rule presented in Fig. 3a *)
    r ~ x;
Add(r, x) >< S(y) => (* Inpla variant of rule presented in Fig. 3b *)
    Add(r, S(x)) ~ y;

(* (1 + 0) + (1 + 0): two active pairs *)
Add(x, S(Z)) ~ Z,
Add(Add(r, x), S(Z)) ~ Z;
(* Printing the result and freeing the `r` name *)
r; free r;
```

Listing 1: Peano numerals addition in Inpla

Inpla — mature enough, demonstrates good performance and scalability. Interaction-nets-based language + parallel VM.

Vine

IVY, IVM. Differences with Inpla: fixed set of agents (and respective rewriting rules). Vine — high-level language (Rust-like) + interaction-nets-based intermediate representation + parallel virtual machine.

HVM: fixed set of agents (and respective rewriting rules). Bend. Acceleration in Cuda. [3] HVM — high-level language (JS-like) + interaction-nets-based intermediate representation + parallel virtual machine (+GPGPU?).

Other projects like !!! but no active development or instable, or not mature enough.

No information is available on the evaluation or comparison of Vine and HVM. Typical benchmarks for these systems include Fibonacci, Ackermann, sorting, and list traversal. No graph analysis algorithms while such algorithms looks grate candidates for interaction nets.

3. Implementation Details

In this section we describe our implementations: in F# (used as a prototype and reference for other implementations), in Inpla (a low-level language), and in Vine (a high-level language). We present the data structures and functions, list graph analysis algorithms (with short descriptions), and discuss details related to the implementation of the described structures and functions in selected languages.

3.1. Basic Data Structures and Functions

Matrix representation, as described in the F# code, is shown in Listing 2. Our representation is generic, meaning it is parameterized by the matrix cell type. The structure does not only work for matrices with sizes that are powers of two. A user-defined matrix of arbitrary size will be aligned to a square storage with size equal to a power of two using `Dummy` blocks. Since the representation is compressed, the `Storage` stores the real size (as a power of two) together with the respective tree. Finally, the `SparseMatrix` stores the matrix dimensions and the number of non-zero entries (nnz) together with data represented as `Storage`. Data structures for a sparse vector are described in a similar way.

We propose to use `Option<'T>` to explicitly represent cells that have no values. This approach is natural for graph representation: a cell of the adjacency matrix of a graph with edge labels of type `'T` has type `Option<'T>`: either there exists an edge with a label of the respective type (`Some(l)`), or

[3] Issues, problems, eg with QTree

```
type treeValue<'value> = | Dummy | UserValue of 'value

(*
| x1 | x2 |
----------
| x3 | x4 |
*)
type qtree<'value> =
    | Node of qtree<'value> * qtree<'value> * qtree<'value> * qtree<'value>
    | Leaf of treeValue<'value>

[<Struct>]
type Storage<'value> =
    // Storage is always size-x-size square.
    val size: uint64<storageSize>
    val data: qtree<'value>
    new(_size, _data) = { size = _size; data = _data }

[<Struct>]
type SparseMatrix<'value> =
    val nrows: uint64<nrows>
    val ncols: uint64<ncols>
    val nvals: uint64<nvals>
    val storage: Storage<Option<'value>>
```

Listing 2: Matrix representation

there is no edge at all (`None`). Thus, for a sparse matrix with filled cells of type `'T`, the storage has type `Option<'T>`.

We implemented a minimal set of operations over matrices and vectors required to implement selected graph analysis algorithms. The list of basic functions includes `map`, `fold`, `reduce`. Element-wise operations are based on `map2`. Particularly, `mask` is implemented as a case of `map2`. Linear algebra functions include: `vxm` (generic vector-matrix multiplication) and `mxm` (generic matrix-matrix multiplication). We also support transpose and extracting the triangular part of a matrix (required for the triangle counting algorithm). We did not implement fused versions. Conversion to and from coordinate list format is provided to simplify input graph building. All operations take sparsity into account and preserve compression. No manual parallelization to !!!

Exception handling is unlikely to be possible in interaction nets in the traditional way due to their highly parallel and irregular nature. Moreover, the selected languages (Inpla and Vine) do not provide any specific mechanisms for exception handling. We focus on realistic code, so we should handle not only correct cases but also erroneous ones (e.g., inconsistent matrix or vector sizes). For this, we use the `Result` type (even in the F# version)[4] that explicitly represents two cases: `Ok<'R>` for successfully completed computations and `Err<'E>` for errors. This way of handling errors is represented in many languages such as Rust, F#, etc.

3.2. Implemented Graph Analysis Algorithms

We implemented three basic graph analysis algorithms.

[4]We do this to be as close to interaction-net-based implementations as possible. The F# language has traditional exception mechanisms.

- Level-based BFS: for the given graph and start vertex, it returns a sparse vector `result` such that if `result[i] = Some(k)`, then vertex `i` was visited at step `k`. If `result[i] = None`, then `i` is unreachable.
- SSSP: for the given graph and start vertex, it returns a sparse vector `result` such that if `result[i] = Some(k)`, then the shortest path from the start vertex to `i` has length `k`. If `result[i] = None`, then `i` is unreachable. To prevent an infinite loop in the case of negative cycles, we check that the number of iterations should not be greater than the number of vertices in the input graph.
- Triangle counting algorithm.

All algorithms were implemented using operations over matrices and vectors similarly to how they are described in GraphBLAS-related materials. We provide basic implementations without any optimizations such as the *push-pull* optimization for BFS or SSSP.

3.3. Interaction Nets Related Details

The translation is almost "as-is" without any application of interaction-net-specific optimizations or code templates.

In general, a high-level programming language function is translated into an interaction nets agent with a single port and an associated rewriting rule. Calling such a function involves connecting the function agent to its argument (which may be represented as a tuple agent). The result is then obtained on the agent's port after a sequence of rewritings. This defines an interface function, which is typically rewritten into a single implementation agent with more complex interaction rules.

```
// Simple interaction rule; real implementation omitted
SMMap2(r) >< (f, m1, m2) =>
    SMMap2Decons_1(r, f, m2) ~ m1;
// ...
// Operations on generic data - also agents and rules
op_add(r) >< (x, y) =>
    op_add_1(r, x) ~ y;

// Calling the interface function with real data
m1 ~ SparseMatrix(4, 4, 16, 4, QTreeLeaf(UserValue(Some(1)))),
m2 ~ SparseMatrix(..., QTreeNode(...)),
SMMap2(r) ~ (%op_add, m1, m2);
r; free r;
```

Listing 3: Interface function pattern in Inpla

Inpla has no I/O operators, so input data structures must be generated in advance. Error handling is minimal and relies on the virtual machine's "no interaction rule" exception for unforeseen situations, as implementing comprehensive checks would introduce significant cognitive overhead in such a low-level language.

A significant concern is memory consumption. Interaction nets programming requires the use of duplicator agents for data reuse. Inpla's current implementation eagerly copies entire subnets when duplicating, resulting in substantial memory overhead compared to the F# prototype. Addressing this issue requires further investigation.

Vine supports reading. Execution is parallel, so separating concerns is difficult. Error handling uses the standard `Result` type from the Vine language.

Inpla does not support floats. Vine supports floats but without comparison (possibly due to problems with NaN comparison). Therefore, weights in SSSP are represented as integers, not floats, in both Vine and Inpla implementations.

4. Evaluation

The goal of this evaluation is to assess the applicability of runtimes based on Interaction Nets to real-world problems, identify bottlenecks and issues, and highlight directions for further investigation.

4.1. Experimental Setup

Hardware. All experiments were conducted on a system with an AMD Ryzen 7 5800X 8-core processor (16 threads with hyper-threading enabled, base clock 3.8 GHz) and 128 GiB of DDR5 RAM.

Software. We compare five implementations presented below.

- **F#**: Our quadtree-based single-threaded implementation, compiled for the .NET 10.0 runtime. This serves as a baseline representing a conventional functional language approach and prototype for porting to Interaction Nets. BenchmarkDotNet[5][**?**] for benchmarking automation (e.g. fro JIT warmup).
- **Inpla**: Our interaction-net-based implementation in Inpla low-level language, executed on the Inpla runtime system (version 0.13.4, patched). Our patches of Inpla allow one to use arbitrary heap size configuration with per-thread adjustments via scripting. The patch also increases several virtual machine compile-time constants to support very large trees (thus, large graphs). Additionally, a bottleneck in lists parsing was discovered during testing, and the fix has since been merged into the main Inpla repository[6]. In this evaluation we use 120 GB of heap. Three or five runs depending on data size.
- **Vine**: Our interaction-net-based implementation in Vine high-level language, executed on the Vine runtime system[7]. We set 110 GB of heap per worker. Five runs for each configuration
- **LAGraph**: A collection of graph algorithms built on SuiteSparse:GraphBLAS (version !!!), providing state-of-the-art, linear-algebra-based reference implementations of BFS, SSSP, and triangle counting. We use the provided algorithms as a SOTA baseline for high-performance parallel graph analysis. Also we use LAGraph as a baseline to analyze scaling across available threads. Number of runs !!!
- **NetworkX**: A popular Python library for graph analysis (version !!!). We use the algorithms implemented in NetworkX as a well-known baseline. We do not use any special backends for NetworkX or special settings. Number of runs !!!

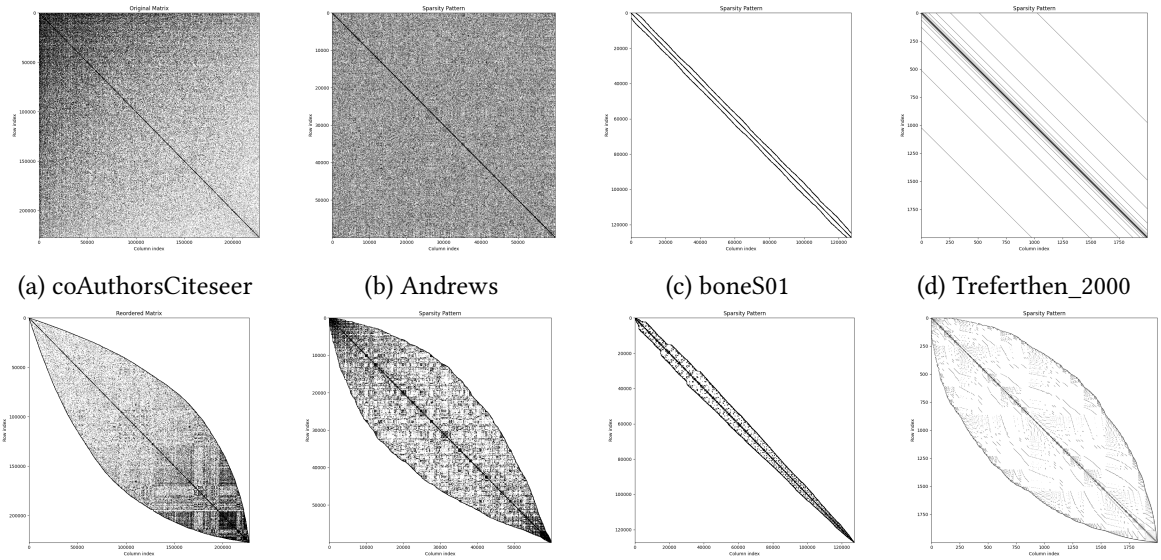
Dataset. We evaluate our implementations on several real-world and synthetic graphs from the SuiteSparse Matrix Collection [12]. The graphs are chosen to represent a variety of sizes, sparsity patterns, and application domains, taking into account the capabilities and limitations of the selected competitors. Table 1 summarizes number of vertices ($|V|$) and number of edges ($|E|$) of selected graphs. Weights were corrected by shifting all weights with respect to minimal weight over particular graph to prevent negative loops, and converted to `Int` due to Inpla does not support floats.

Reordering. Patterns reordering 4. *coAuthorsCiteseer* reordered only We can see, that some graphs.... initial is diagonal parallel...

[5] https://github.com/dotnet/BenchmarkDotNet
[6] https://github.com/inpla/inpla/pull/4
[7] !!!

Table 1
Dataset properties

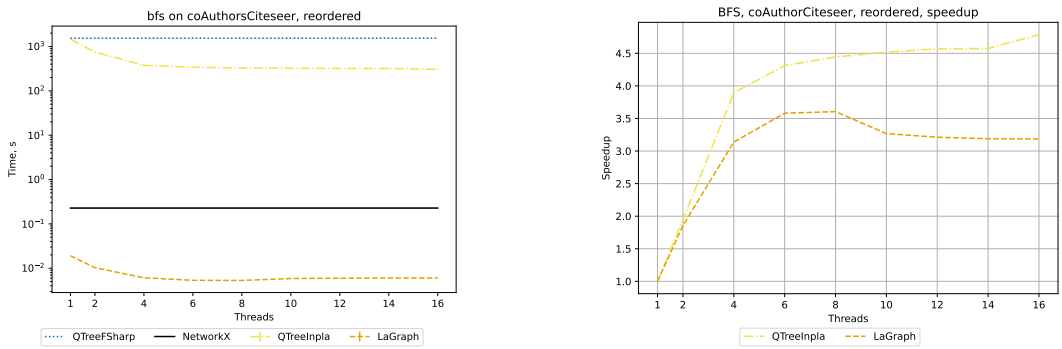
Graph	$\lvert V \rvert$	$\lvert E \rvert$
coAuthorsCiteseer	227,320	1,628,268
boneS01	127,224	5,516,602
usroads-48	126,146	323,900
Andrews	60,000	760,154
Trefethen_2000	2000	41,906
gr_30_30	900	7,744



(a) coAuthorsCiteseer (b) Andrews (c) boneS01 (d) Treferthen_2000

(e) coAuthorsCieseer (f) Andrews (g) boneS01 (h) Treferthen_2000 reordered

Figure 4: Matrices shapes: original and after reordering



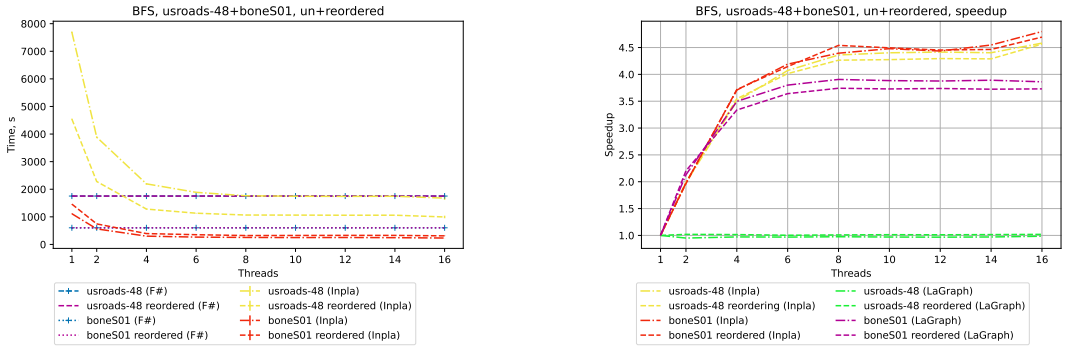
(a) Execution time (b) Scaling

Figure 5: Evaluation results for BFS on *coAuthorsCiteseer* (reordered)

Research Questions. Our evaluation addresses the following research questions:

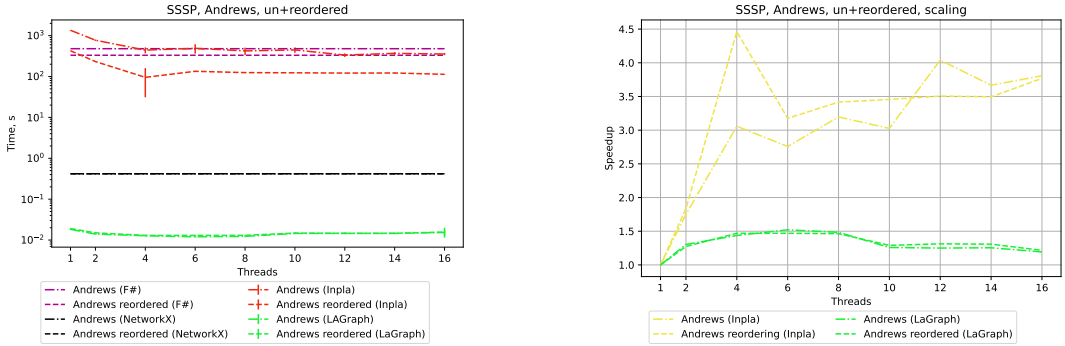
RQ1 (Correctness): Do the F# and Inpla implementations produce correct results for BFS and triangle counting on all datasets, when compared against reference outputs from NetworkX and LAGraph?

RQ2 (Performance – Inpla vs. Vine vs. State-of-the-Art): How does the execution time of Inpla compare to LAGraph on BFS, SSSP, and triangle counting? We also include F# and NetworkX as additional baselines representing different points in the performance–productivity trade-off.



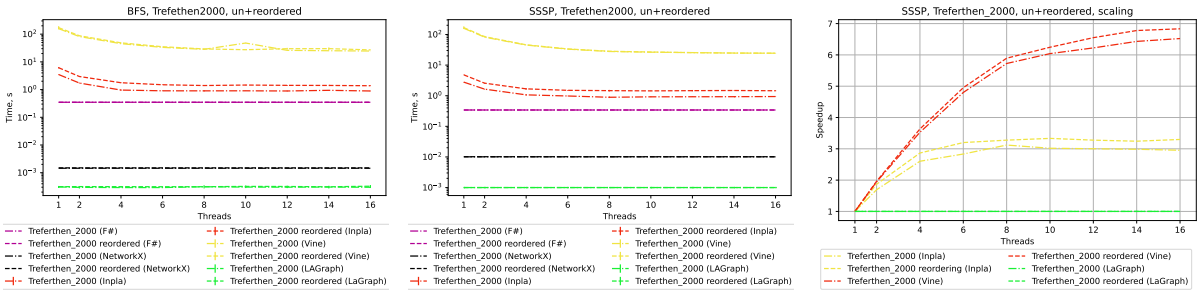
(a) Execution time (b) Scaling

Figure 6: Evaluation results for BFS on *bomeS01* and *usroads-48* (both original and reordered)



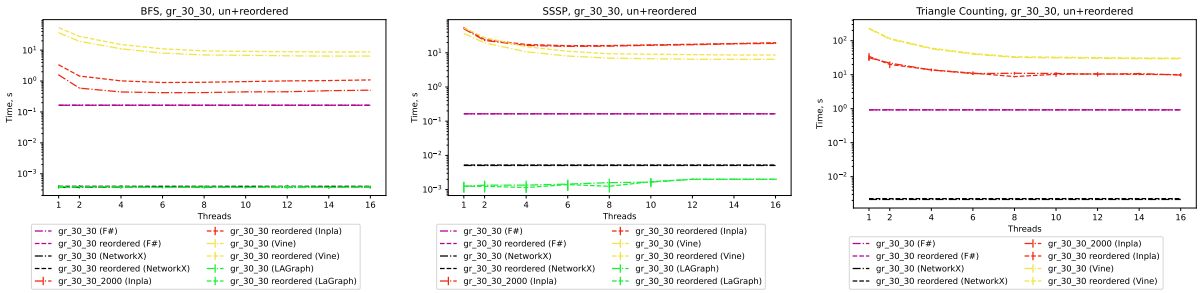
(a) coAuthors reordered (b) Scaling

Figure 7: SSSP evaluation results for *Treferthen_2000* (both original and reordered)



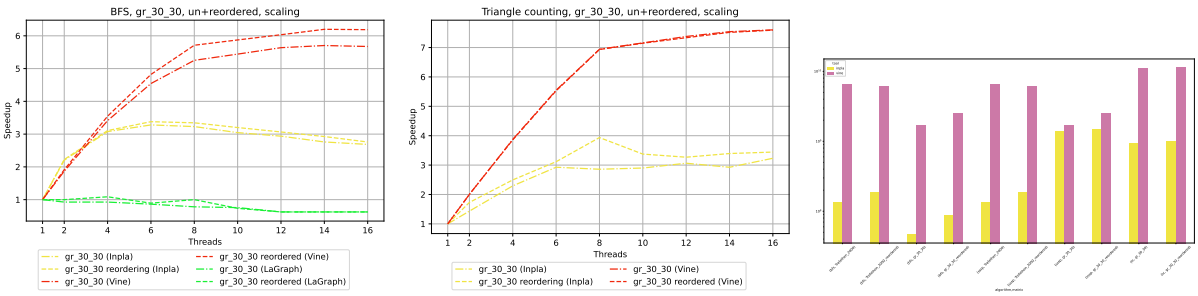
(a) BFS (b) SSSP (c) SSSP scaling

Figure 8: Evaluation results for *Treferthen_2000* (both original and reordered)



(a) BFS (b) SSSP (c) Triangle counting

Figure 9: Evaluation results for *gr_30_30* (both original and reordered)



(a) BFS scaling (b) Triangle counting scaling (c) Total number of interactions

Figure 10: Inpla and Vine comparison

RQ3 (Parallel Scaling): How well do the Inpla, Vine, and LAGraph implementations scale with the number of available CPU cores? We measure speedup from 1 to all available cores for BFS, SSSP, and triangle counting, with hyper-threading both enabled and disabled.

RQ4 (Impact of Matrix Reordering): Matrix reordering techniques (e.g., reverse Cuthill–McKee) can improve locality and cache performance.

Even for [13] How does applying such reordering affect the performance of Inpla, F#, and LAGraph implementations?

4.2. Evaluation Results

Graphics, tables.

Reordering: figures with sparsity patterns

Analysis.

- With growth of grapj size Inpla dominates F# more and more.
- Vine is slow.

Technical. Instability of Inpla. Data races. memory management.

5. Conclusion and Future Work

We have demonstrated a feasible mapping of GraphBLAS concepts to interaction nets using quadtrees, provided concrete implementations in Inpla and Vine, and evaluated their scalability. The performance gap compared to state-of-the-art parallel linear-algebra-based frameworks remains substantial, and scalability is limited.

Languages: But for comfortable usage `Result` monad or similar mechanisms is required.

Several bottlenecks and limitations were identified. The naive implementation suffers from a duplication problem: a huge number of duplicated nodes limits both performance and memory efficiency. Reordering (RCM) is crucial for performance and memory consumption but is not handled automatically. Currently it is a separated preprocessing step. Moreover, only a limited set of operations and algorithms has been evaluated.

Future work includes investigating improved duplication strategies. The set of implemented operations and algorithms should be extended to include indexed operations, slices, and such graph analysis algorithms as PageRank, minimum spanning forest (MSF), regular path queries, context-free path queries (CFPQ), and maximal independent set (MIS).

A systematic evaluation of other interaction-net-based implementations, such as HVM, is also required. Alternative sparse matrix representations, such as the coordinate list (COO) format, rely on sorting—an operation that parallelizes well in interaction nets and therefore warrants further investigation.

Classical implementations of algorithms. May interaction nets be good enough to parallel straightforward implementations?

Automatic kernel fusion presents another direction. Advanced optimization techniques natural to functional programming, such as supercompilation [14] or distillation [15], could be applied to automatically fuse sequences of operations, analogous to manual fusion for masks and other constructs in classic GraphBLAS. Partial evaluation for interaction nets [16] Whether these techniques can be adapted for interaction nets remains an open question.

Finally, hardware acceleration remains an open opportunity. Designing an FPGA specifically optimized for interaction net reduction could exploit the fine-grained parallelism inherent to the model and yield substantial performance gains.

6. Code Availability Statement

All implementations, resources and scripts for evaluation available at GitHub.

- F# prototyping code: https://github.com/Lamagraph/QTreeFSharp.
- Inpla implementation: https://github.com/Lamagraph/QTreeInpla.
- Vine implementation: https://github.com/Lamagraph/QTreeVine.
- Patched version of Inpla: https://github.com/Lamagraph/inpla
- Benchmarking scripts and dataset configurations.

Acknowledgments

Thanks to the developers of ACM consolidated LaTeX styles https://github.com/borisveytsman/acmart and to the developers of Elsevier updated LaTeX templates https://www.ctan.org/tex-archive/macros/latex/contrib/els-cas-templates.

Declaration on Generative AI

During the preparation of this work, the author(s) used X-GPT-4 and Gramby in order to: Grammar and spelling check. Further, the author(s) used X-AI-IMG for figures 3 and 4 in order to: Generate images. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.

References

[1] B. Brock, A. Buluç, T. G. Mattson, S. McMillan, J. E. Moreira, Introduction to graphblas 2.0, in: 2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2021, pp. 253–262. doi:10.1109/IPDPSW52791.2021.00047.

[2] T. A. Davis, Algorithm 1037: Suitesparse:graphblas: Parallel graph algorithms in the language of sparse linear algebra, ACM Trans. Math. Softw. 49 (2023). URL: https://doi.org/10.1145/3577195. doi:10.1145/3577195.

[3] T. Mattson, T. A. Davis, M. Kumar, A. Buluc, S. McMillan, J. Moreira, C. Yang, Lagraph: A community effort to collect graph algorithms built on top of the graphblas, in: 2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2019, pp. 276–284. doi:10.1109/IPDPSW.2019.00053.

[4] C. Yang, A. Buluç, J. D. Owens, Graphblast: A high-performance linear algebra-based graph framework on the gpu, ACM Trans. Math. Softw. 48 (2022). URL: https://doi.org/10.1145/3466795. doi:10.1145/3466795.

[5] Y. Hu, Y. Du, E. Ustun, Z. Zhang, Graphlily: Accelerating graph linear algebra on hbm-equipped fpgas, in: 2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD), 2021, pp. 1–9. doi:10.1109/ICCAD51958.2021.9643582.

[6] T. Henriksen, N. G. W. Serup, M. Elsman, F. Henglein, C. E. Oancea, Futhark: purely functional gpu-programming with nested parallelism and in-place array updates, SIGPLAN Not. 52 (2017) 556–571. URL: https://doi.org/10.1145/3140587.3062354. doi:10.1145/3140587.3062354.

[7] R. Leißa, K. Boesche, S. Hack, A. Pérard-Gayot, R. Membarth, P. Slusallek, A. Müller, B. Schmidt, Anydsl: a partial evaluation framework for programming high-performance libraries, Proc. ACM Program. Lang. 2 (2018). URL: https://doi.org/10.1145/3276489. doi:10.1145/3276489.

[8] M. Steuwer, T. Remmelg, C. Dubach, Lift: A functional data-parallel ir for high-performance gpu code generation, in: 2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2017, pp. 74–85. doi:10.1109/CGO.2017.7863730.

[9] Y. Lafont, Interaction nets, in: Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '90, Association for Computing Machinery, New York, NY, USA, 1989, p. 95–108. URL: https://doi.org/10.1145/96709.96718. doi:10.1145/96709.96718.

[10] N. Huber, W. Yi, An encoding of interaction nets in ocaml, Electronic Proceedings in Theoretical Computer Science 417 (2025) 1–16. URL: http://dx.doi.org/10.4204/EPTCS.417.1. doi:10.4204/eptcs.417.1.

[11] I. Mackie, S. Sato, In-place graph rewriting with interaction nets, in: A. Corradini, H. Zantema (Eds.), Proceedings 9th International Workshop on Computing with Terms and Graphs, TERMGRAPH 2016, Eindhoven, The Netherlands, April 8, 2016, EPTCS, 2016, pp. 15–24. URL: https://doi.org/10.4204/EPTCS.225.4. doi:10.4204/EPTCS.225.4.

[12] S. P. Kolodziej, M. Aznaveh, M. Bullock, J. David, T. A. Davis, M. Henderson, Y. Hu, R. Sandstrom, The suitesparse matrix collection website interface, Journal of Open Source Software 4 (2019) 1244. URL: https://doi.org/10.21105/joss.01244. doi:10.21105/joss.01244.

[13] O. Asudeh, S. Mahdipour Saravani, F. Rastello, G. Sabin, P. Sadayappan, How effective is matrix reordering for improving performance of sparse matrix-vector multiplication?, in: Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC Workshops '25, Association for Computing Machinery, New York, NY, USA, 2025, p. 823–827. URL: https://doi.org/10.1145/3731599.3767441. doi:10.1145/3731599.3767441.

[14] V. F. Turchin, Supercompilation: Techniques and results, in: D. Bjørner, M. Broy, I. V. Pottosin (Eds.), Perspectives of System Informatics, Springer Berlin Heidelberg, Berlin, Heidelberg, 1996, pp. 227–248.

[15] G. Hamilton, The next 700 program transformers, in: E. De Angelis, W. Vanhoof (Eds.), Logic-Based Program Synthesis and Transformation, Springer International Publishing, Cham, 2022, pp. 113–134.

[16] D. Béchet, Partial evaluation of interaction nets, in: M. Billaud, P. Castéran, M. Corsini, K. Musumbu, A. Rauzy (Eds.), Actes WSA'92 Workshop on Static Analysis (Bordeaux, France), September 1992, Laboratoire Bordelais de Recherche en Informatique (LaBRI), Proceedings, Series Bigre, Atelier Irisa, IRISA, Campus de Beaulieu, 1992, pp. 331–338.